

ПВП

Пункты Взимания Платы

Разработка информационной системы для пунктов взимания
платы

Выполнил студент группы ПИ-23-2 Суханов Кирилл

Монолитный PHP → Laravel 10 · Миграция и рефакторинг

Предметная область

Система автоматизации платных дорожных пунктов

Сущности

- ▶ **ПВП** — пункт взимания платы (географическая точка)
- ▶ **Полоса** — конкретная полоса движения на ПВП (универсальная, легковая, грузовая...)
- ▶ **Водитель** — зарегистрированный пользователь с балансом
- ▶ **Транспортное средство** — привязано к водителю, имеет тип (легковой/грузовой)
- ▶ **Транспондер** — устройство для бесконтактной оплаты (ESP / E-Pass)

Бизнес-процессы

- ▶ Проезд ТС через полосу → регистрация транзакции
- ▶ Автоматический расчёт тарифа по типу ТС, времени, дню недели
- ▶ Списание средств с баланса водителя или транспондера
- ▶ Автоматическое наложение штрафа при неоплате (триггер БД)
- ▶ Пополнение баланса водителем или администратором
- ▶ Аналитика: выручка, загрузка полос, популярные маршруты

Цель и задачи

Перевод legacy-монолита на современный фреймворк

1

Миграция

Перенести Pure
PHP/PDO-
приложение (≈620
строк в index.php)
на Laravel 10 с
сохранением всей
функциональности

2

Аутентификация

Добавить
регистрацию,
вход по паролю и
разграничение
доступа:
администратор
(полный CRUD)
vs водитель
(профиль)

3

UX

Современный
интерфейс:
Bootstrap 5,
DataTables,
модальные окна,
каскадные
списки,
адаптивная
вёрстка

4

Инфраструктура

Полная Docker-
сборка (PHP 8.2 +
PostgreSQL 16 +
Adminer) в один
docker-compose up

5

Аналитика

Дашборд
администратора,
графики Chart.js,
экспорт в CSV
для отчётов

6

Личный кабинет

Профиль
водителя: авто,
поездки, штрафы,
транспондеры,
пополнение
баланса

Технологический стек

Бэкенд

PHP 8.2

Laravel 10

Eloquent ORM

База данных

PostgreSQL 16

Триггеры

PL/pgSQL

Инфраструктура

Docker Compose

PHP 8.2-cli

Adminer

Фронтенд

Bootstrap 5.3

jQuery 3.6

DataTables 1.13

Chart.js 4

Bootstrap Icons

Архитектура

- ▶ MVC (Model-View-Controller)
- ▶ Session-based аутентификация (без API)
- ▶ Middleware: `auth.driver` + `admin`
- ▶ HTML-over-fetch (модалки с partial-вьюхами)
- ▶ Часовой пояс: Europe/Moscow

Миграция

legacy/ — Pure PHP/PDO (≈620 строк) →
Laravel (13 контроллеров, 52 метода)

Схема базы данных

12 таблиц, связанных внешними ключами

Таблица	Назначение	Записей
drivers	Водители	101
vehicles	Транспортные средства	~300
auto_types	Типы ТС (легковой/грузовой...)	5
payment_points	Пункты взимания платы	~15
lanes	Полосы движения	~40
lane_types	Типы полос	5
tariffs	Тарифы (цена × тип ТС × время)	~50
transponders	Транспондеры (ESP)	~80
transactions	Транзакции проезда	~500
payment_methods	Способы оплаты	4
finest	Штрафы	~30
fine_types	Типы штрафов	5

Ключевые особенности

- ▶ Все Primary Key — именованные
(id_driver , id_vehicle , ...)
- ▶ Триггер
trg_auto_fine — при транзакции со статусом неоплата автоматически создаётся штраф 500Р
- ▶ CHECK-констрейнты на статусах, днях недели
- ▶ Индексы на
phone_number ,
plate_number ,
datetime
- ▶ Изначальная схема — db/init.sql

(развёртывается
через Docker)

- ▶ Laravel-миграции дублируют схему для удобства разработки

5 / 12

Аутентификация и ролевая модель

Два уровня доступа

Администратор

driver_id = 1 (Сергей Петров,
+70066564484)

→ полный доступ: все CRUD,
дашборд, статистика,
управление балансом

Водитель

Все остальные
зарегистрированные
пользователи

→ только профиль: просмотр
авто, поездок, штрафов,
пополнение баланса

Реализация

- ▶ **Вход:** по номеру телефона + пароль (bcrypt)
- ▶ **Сессия:**

```
session(['driver_id' => $id,  
'is_admin' => $id === 1])
```
- ▶ **Middleware:**
 - ▶ `AuthenticateDriver` — проверяет наличие `session('driver_id')`
 - ▶ `AdminOnly` — проверяет `session('is_admin')`
- ▶ **Регистрация:** ФИО + телефон + пароль (с

подтверждением)

- ▶ **Нормализация телефона:**
+7, 8XXX, XXX →
+7XXXXXXXXXXXX

Маршруты приложения

Всего ~40 именованных маршрутов, 13 контроллеров, 52 метода

Публичные

Метод	URI
GET	/login
POST	/login
GET	/register
POST	/register
GET	/logout

Аутентифицированные

Метод	URI
GET	/profile
POST	/profile/balance

Администраторские (Middleware: auth.driver + admin)

URI	Сущность
/	Дашборд
/drivers/*	Водители (9 маршрутов)
/vehicles/*	Транспорт (5 маршрутов)
/payment-points/*	ПВП + Полосы (9 маршрутов)
/transactions/*	Транзакции (3 маршрута)
/fines/*	Штрафы (4 маршрута)

/tariffs/*

Тарифы (4 маршрута)

/transponders/*

Транспондеры (5 маршрутов)

/statistics/*

Статистика (3 маршрута)

7 / 12

Реализованные функции



Управление ПВП

Добавление/
редактирование
пунктов оплаты,
полос с типами и
статусами,
привязка тарифов



Транспорт

Регистрация ТС
по госномеру,
привязка к
водителю и типу
авто,
транспондеры



Транзакции

Каскадные списки
(ПВП → полоса),
авторасчёт суммы
по тарифу,
поддержка
транспондера



Штрафы

Ручное и
автоматическое
(триггер БД)



Статистика

Дашборд (30
дней), 4 графика
Chart.js, CSV-



Личный кабинет

Профиль
водителя: список
авто, история

наложение
штрафов, отметка
оплаты

экспорт с UTF-8
BOM

поездок, штрафы,
пополнение
баланса

8 / 12

Трудности и решения

♦ Миграция с legacy на Laravel

Исходное приложение — один PHP-файл с PDO-запросами и HTML-вставками. Весь код пришлось декомпонировать на 13 контроллеров, 12 моделей, 17 шаблонов.

Решение: поэтапный перенос — сначала настройка Docker, затем маршруты, контроллеры, модели, Blade-шаблоны. Каждый этап проверялся отдельно.

♦ Аутентификация без стандартного Auth

Не использовали Laravel Breeze/Jetstream. Вход по номеру телефона (не email), пароль хранится в `drivers.password`, а не в `users`.

Решение: кастомный AuthController с ручной проверкой `Hash::check()`, сессионная аутентификация, два middleware.

♦ Одинаковая структура БД

Таблицы используют именованные PK/FK (`id_driver`, `id_vehicle`), что требует явного указания

♦ PHP built-in server + Docker

Laravel работает через `php artisan serve` (однопоточный dev-сервер).

`primaryKey` в каждой модели. Некоторые колонки в migrations не совпадают с реальной схемой.

Решение: адаптировали код под существующую БД (init.sql), а не под миграции. Добавили

`LaneType` модель для FK `id_lane_type`.

Пришлось настроить

`SESSION_DRIVER=file` и корректные права на `storage/`.

Решение: создание

`bootstrap/cache/` в entrypoint, anonymous volume для vendor, явная установка timezone.

♦ Русский язык и кодировки

Проблемы с UTF-8: DataTables с русской локалью, CSV для Excel (UTF-8 BOM), корректный сброс кэша шаблонов.

Решение: `APP_LOCALE=ru`,

BOM-маркер в CSV, явная

`@php` директива для моделей в шаблонах.

9 / 12

Интерфейс и UX



Вход / Регистрация

Форма входа с маской телефона (+7 (XXX) XXX-XX-XX), паролем, ссылкой на



Дашборд администратора

Сводка за 30 дней: количество транзакций, выручка, уникальные водители, средний чек. Последние 5 транзакций.



Профиль водителя

Аватар, ФИО, баланс, список автомобилей, таблицы

регистрацию.
Список тестовых
аккаунтов для
отладки.

поездов и
штрафов.
Кнопка
пополнения с
быстрыми
суммами.



DataTables

Все CRUD-
страницы
используют jQuery
DataTables: поиск,
сортировка,
пагинация, русский
язык. Модальные
окна для создания/
редактирования.



Каскадные списки

При создании транзакции:
выбор ПВП → AJAX
подгружает полосы →
автоматически
заполняется сумма по
тарифу.



Графики Chart.js

Линейный
(выручка по
дням), пончик
(способы
оплаты),
горизонтальные
бары (ТОП
точек и
водителей).

10 / 12

Docker и развёртывание

docker-compose.yml

```
services:  
  app: ← PHP 8.2 CLI +
```

Процесс сборки

1. ▶ `docker-compose up -d` —
запуск всех сервисов

```
artisan serve  
  pvp_db: ← PostgreSQL 16 +  
init.sql  
  adminer: ← Web UI для БД  
(порт 8080)
```

2. ▶ PostgreSQL
инициализируется скриптом
`db/init.sql`
3. ▶ PHP-контейнер выполняет
`composer install`,
генерирует ключ
4. ▶ `php artisan serve` на порту
8000
5. ▶ Готово: localhost:8000

Приложение полностью самодостаточно
— не требует установки PHP, Composer
или PostgreSQL на хосте.

11 / 12

Результаты

13

Контроллеров
52 метода

12

Моделей Eloquent
+ связи и
аксессуары

17

Blade-шаблонов
+ layout + модалки

40

Маршрутов
с middleware-
защитой

12

Таблиц в БД
101 водитель, ~500
транзакций

3

Docker-сервиса
App + DB + Adminer

Что сделано

- ▶ Миграция монолитного PHP-приложения на Laravel 10
- ▶ Система аутентификации: вход/регистрация/роли
- ▶ Полный CRUD для всех сущностей через DataTables + модальки
- ▶ Дашборд администратора и детальная аналитика с графиками
- ▶ Личный кабинет водителя с пополнением баланса
- ▶ Docker-контейнеризация (одна команда для запуска)

Перспективы

- ▶ Переход на Nginx + PHP-FPM для production
- ▶ REST API для мобильного приложения
- ▶ Интеграция с реальными платёжными шлюзами
- ▶ WebSocket-уведомления о проездах и штрафах
- ▶ Тесты (PHPUnit + Dusk для E2E)
- ▶ GitHub Actions CI/CD